BigTech Electronics Store

Project Report

Prepared by: Harika

Abstract

This report presents a student electronics store with 28 synthetic products across seven categories, website-specific Ezzie support, deterministic order rules, and a Java Spring Boot backend. The project covers browsing, cart calculations, checkout, stock changes, payment failure, confirmation, delivery estimation, and order history.

Ten automated tests pass across JavaScript and Java. Five experiments document test coverage, catalogue scale, failure scenarios, browser evidence, and backend separation. All commercial data and payment outcomes are synthetic.

Keywords: electronics retail; JavaScript; Spring Boot; inventory; payment failure; testing

Report structure

The report presents the problem, source data, preparation method, evaluation design, five evidence-based experiments, limitations, reproducibility details, and conclusion.

1. Project overview and problem statement

BigTech is a student electronics store that connects catalogue browsing, cart totals, checkout, payment outcomes, delivery estimates, order history, and Ezzie customer support. I built it to demonstrate that a small retail project can still verify both successful and failed order behaviour.

The catalogue contains 28 synthetic products across seven electronics categories. A Java Spring Boot service provides the canonical order coordinator, while the browser interface remains deployable as a simple static site.

2. Architecture and boundaries

The frontend uses HTML, CSS, and modular JavaScript. Catalogue and browser state are kept separate from calculations and order rules. Ezzie answers only BigTech questions and returns verified in-stock suggestions.

The Java service separates inventory, payment, fulfilment, and coordination responsibilities. All orders and payment outcomes are simulated. There is no authentication, shared production database, real payment provider, or courier integration.

Component	Responsibility
Inventory service	Validate requested quantity
Payment service	Return success or failure outcome
Fulfilment service	Calculate estimated delivery
Order coordinator	Apply sequence and stopping rules
Ezzie	Website-specific product and order support

3. Verification method

Seven Node tests cover catalogue availability, totals, payment failure, assistant scope, budget recommendations, category coverage, and gaming intent. Three Java tests cover successful confirmation, stock rejection before payment, and payment failure without a delivery date.

Browser verification records the storefront, cart and checkout, and Ezzie assistant. These tests make business behaviour inspectable instead of relying only on a visual walkthrough.

4. Experiment 1: automated test suites

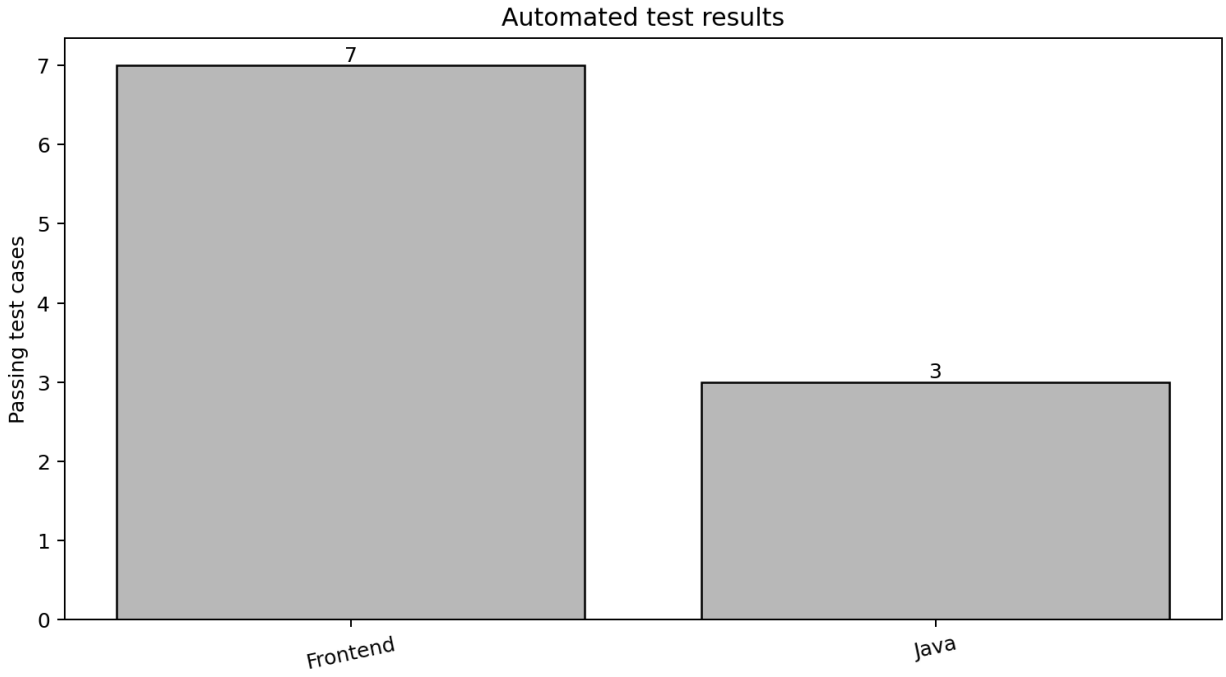

Figure 1. Passing frontend and Java test cases.

What I tested: Core catalogue, calculation, assistant, stock, payment, and fulfilment rules.

What the graph shows: Seven frontend tests and three Java tests pass.

Conclusion: The project verifies behaviour in both the browser logic and backend coordinator.

5. Experiment 2: catalogue coverage

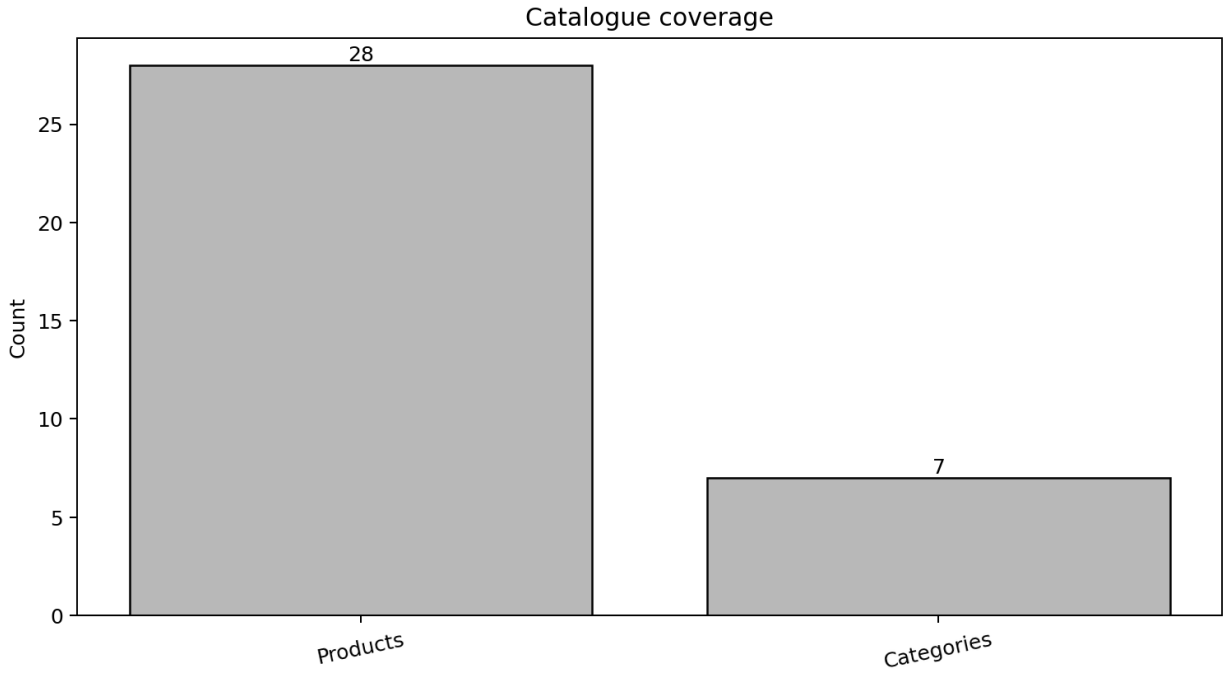

Figure 2. Product and category counts in the synthetic catalogue.

What I tested: Whether the catalogue contains the intended breadth without becoming unnecessarily large.

What the graph shows: Twenty-eight products cover seven electronics categories.

Conclusion: The scope is sufficient for search and recommendation scenarios while remaining easy to review.

6. Experiment 3: order scenarios

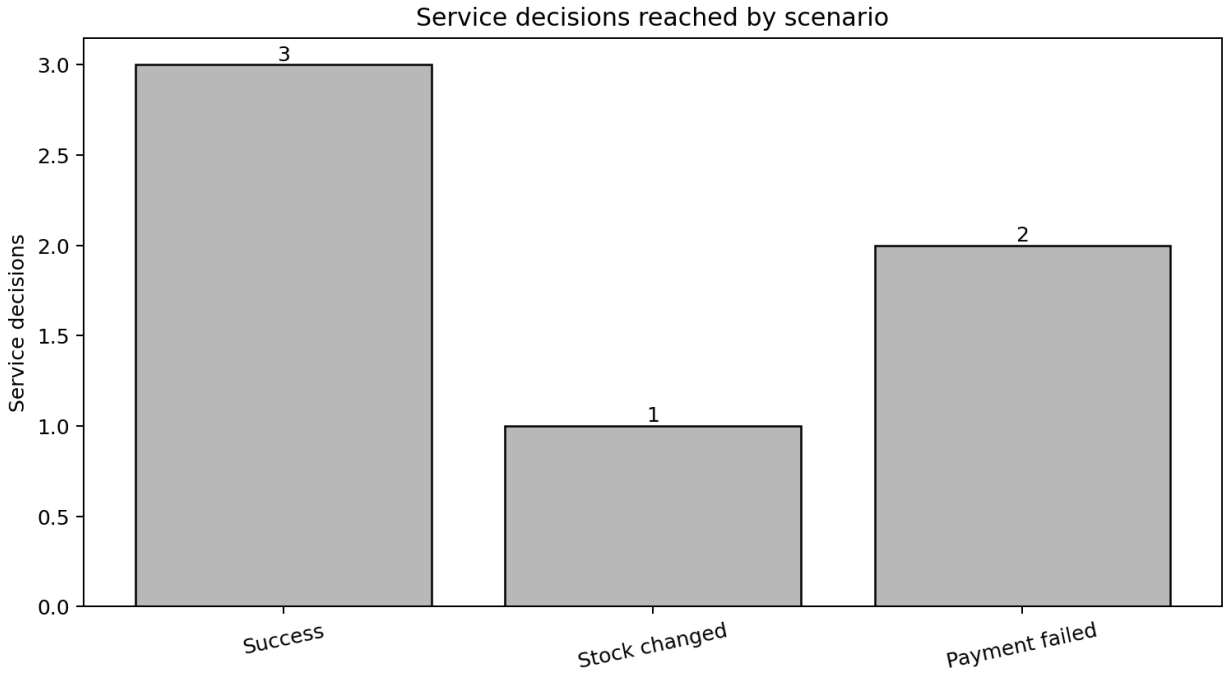

Figure 3. Relative workflow progress for three order outcomes.

What I tested: Whether the coordinator stops before downstream actions after stock or payment failure.

What the graph shows: A successful order reaches fulfilment, stock failure stops before payment, and payment failure produces no delivery estimate.

Conclusion: The Java flow preserves ordering rules and avoids creating a false delivery promise after failure.

7. Experiment 4: browser evidence

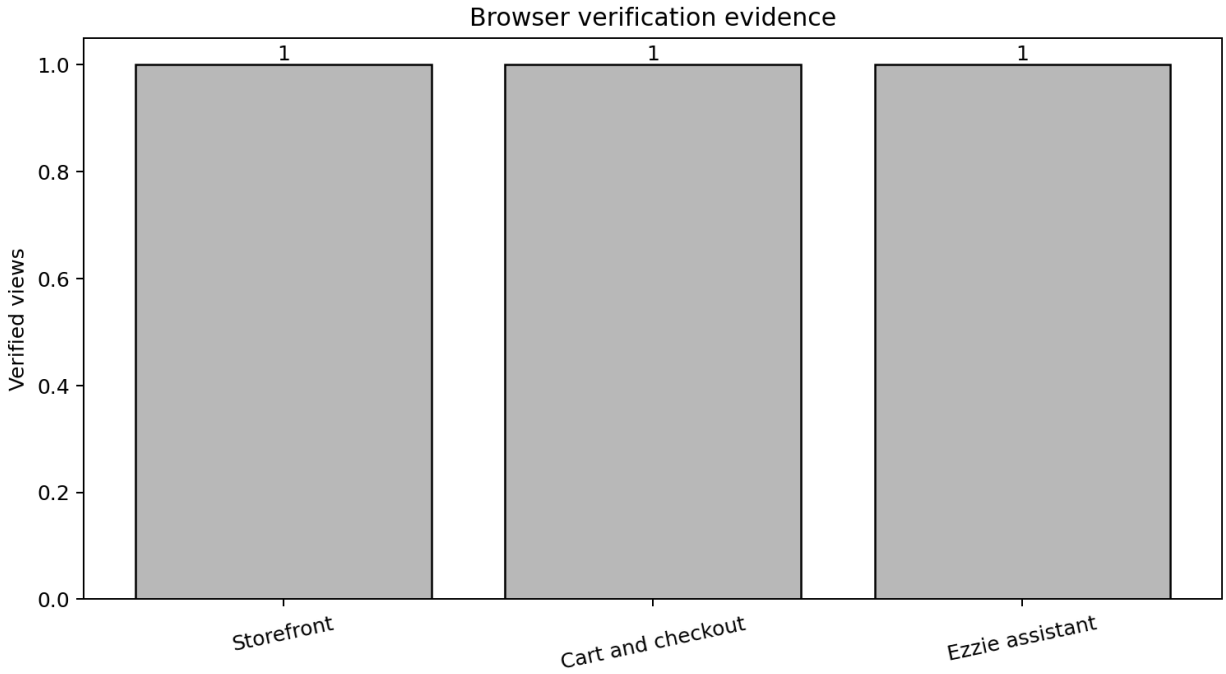

Figure 4. Three committed browser-verification views.

What I tested: Storefront rendering, cart and checkout behaviour, and Ezzie support.

What the graph shows: Each principal customer-facing view has a committed verification capture.

Conclusion: The public interface is supported by observable evidence in addition to unit tests.

8. Experiment 5: backend component boundaries

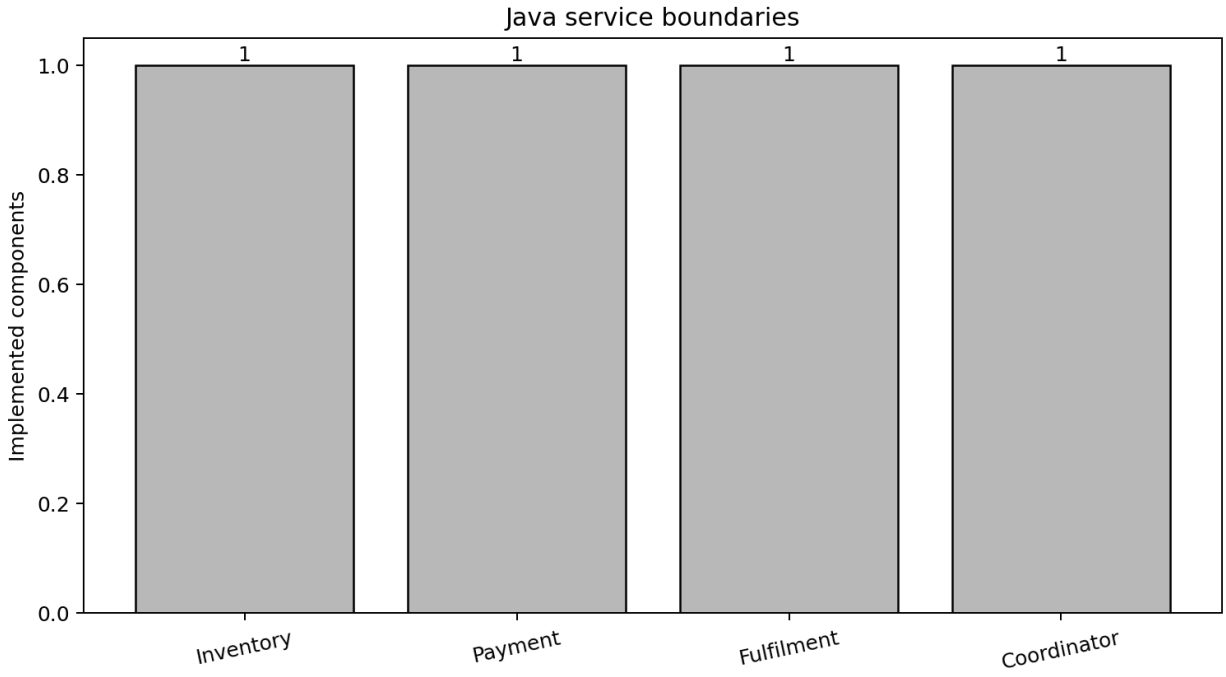

Figure 5. Implemented Java service boundaries.

What I tested: Whether order responsibilities are separated rather than placed in one controller.

What the graph shows: Inventory, payment, fulfilment, and coordination are implemented as distinct components.

Conclusion: The design is small but provides a clear structure for explaining backend control flow.

9. Interface test evidence

The storefront capture below verifies product grouping, INR pricing, search, account access, cart visibility, and the overall electronics identity. Separate captures document checkout and the Ezzie support panel.

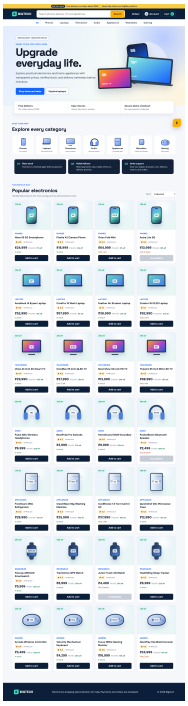

Figure 6. Browser verification of the deployed BigTech storefront.

What I tested: Whether the main catalogue is understandable and usable as the entry point to the order flow.

What the image shows: The interface presents categories, products, prices, stock state, search, cart, and account navigation.

Conclusion: The storefront is functional evidence, while the automated tests remain the source of business-rule verification.

10. Limitations and reproducibility

The data and all customer flows are synthetic. Browser persistence is device-local, and the static deployment does not make the Spring Boot service publicly available. Security, database transactions, shared inventory, payment webhooks, and courier tracking are outside scope.

The repository contains frontend source, Java service code, tests, browser evidence, five evaluation figures, architecture notes, and this report. Future work should deploy the backend, add a persistent order store, use idempotency keys, and verify full integration with a hosted test environment.

11. Conclusion

BigTech demonstrates a complete but understandable electronics-shopping flow. Its value is not only the visible store; it is the tested relationship between catalogue availability, payment outcome, fulfilment, and customer support. The project presents frontend and Java skills without claiming production capabilities that are not implemented.